

LOHEN: Layer-wise Optimizations for Neural Network Inferences over Encrypted Data with high Performance *or* Accuracy

Kevin Nam¹, Youyeon Joo¹, Dongju Lee¹, Seungjin Ha¹, Hyunyoung Oh², Hyungon Moon^{3,*}, and Yunheung Paek^{1,*}

¹Dept. of Electrical and Computer Engineering & ISRC, Seoul National University

²Dept. of AI-Software, Gachon University

³Dept. of Computer Science and Engineering, UNIST

{kvnam, yyjoo, djlee, sjha}@sor.snu.ac.kr, hyoh@gachon.ac.kr, hyungon@unist.ac.kr, ypaek@snu.ac.kr

Abstract

Fully Homomorphic Encryption (FHE) presents unique challenges in programming due to the contrast between traditional and FHE language paradigms. A key challenge is selecting optimal ciphertext configurations (CCs) to ensure security, performance, and accuracy in FHE applications. Optimal CC selection can be complex and labor-intensive. Thus, the conventional practice is settling down on a suboptimal solution of globally choosing one CC that induces acceptable performance. When FHE is applied to neural networks (NNs), the distinct layered architecture of NN models opens the door for a *layer-wise* optimization approach, as an alternative to the conventional practice, which selects locally optimal CCs for individual layers tailored to their structural properties. Sadly, this approach incurs significant overhead from CC switching between layers during NN inference. This paper introduces *LOHEN*, a technique crafted to attain high performance of NN inference by optimizing layer-wise CC selection to minimize the increased overhead incurred by CC switching. Not only that, LOHEN is engineered to attain high accuracy of NN inference, yet delivering performance comparable to state-of-the-art studies, by adopting a layer-wise application of multiple FHE schemes to the target NN. Through two experimental settings, we exhibit LOHEN’s capability that allows developers to customize layer-wise optimizations to adjust the desired levels of performance and accuracy of NN inference over encrypted data, subject to their demands. In the first experiment where we adopt CKKS as our FHE scheme, LOHEN improves inference performance of various NN models by $1.08\text{--}2.88\times$. In the second where we employ two schemes (CKKS+TFHE), LOHEN ensures zero accuracy loss in inference, yet improving performance by $1.34\text{--}1.59\times$ compared to an optimized multi-scheme baseline.

1 Introduction

Fully Homomorphic Encryption (FHE) [1] is one of the known privacy-preserving techniques that enable secure computation on encrypted data. To run an application using FHE, programmers first need to generate an FHE-based code that produces an output identical to that of the original (unencrypted) application code. Unfortunately, building secure and efficient FHE-based code poses unique challenges due largely to the differences between traditional programming paradigms and FHE’s computation model [2]. One key challenge is selecting a set of appropriate *ciphertext configurations* (CCs) for use during computations. This task involves a range of factors, such as vectorization and ciphertext parameters, which are all crucial in determining not only the security strength but also the overall performance and accuracy of the generated FHE code [3, 4]. To determine the best CC for efficient FHE code execution, programmers must be able to consider these factors based on the structural properties of the original code. For example, if the original code contains SIMD (Single Instruction, Multiple Data) operations, a programmer tries to find an optimal packing option that aligns with the data structure and the inherent parallelism of the code in a way to maximize the benefits of SIMD computation [5].

Unfortunately, as the code size increases, the complexity of selecting the optimal CC also escalates significantly. Large codes typically consist of multiple subsections, each with its unique programming structures and properties. The presence of such mutually distinct structures and properties of subsections within one large code makes it extremely complicated, or even impossible, to identify all CCs which together ensure the globally optimal performance of the target FHE code. Thus for practicality, a conventional practice in prior work has been resorting to a suboptimal solution, which involves choosing one reasonably good CC that provides acceptable (although not the best) performance across the entire code. For example, consider a program that includes a subroutine *F* performing matrix-vector multiplication (MVM). In FHE code, *F* would be optimally computed with a CC whose batch

* co-corresponding authors

This paper is accepted for USENIX Security 25

size matches the dimensions of the MVM. However, if the program also has other subroutines dealing with smaller dimensions of data, the conventional approach may adopt a CC targeted to a larger batch size for a greater cause [6]. This choice, while increasing computation latency for F alone, may enhance the overall performance of the program.

With the rapid growth of Machine Learning (ML) as a Service and increasing demands for privacy-preserving ML, researchers are actively exploring the use of FHE for neural network (NN) models over encrypted data [3, 4, 7–10]. In doing so, they often adopt the aforementioned approach, selecting a CC that, while not optimized for each NN layer’s computation within the model, still enables near-optimal performance for the entire NN inference process. However, it is noteworthy that a key difference of NN models from ordinary applications lies in their programming structure. The computation within NNs is structured into clearly divided and concatenated subsections (i.e., layers), each with its unique node structures and interconnections tailored for specific computational purposes. This unique layered structure in NN computing opens the door to another approach, we refer to as *layer-wise optimization*, as an alternative to traditional approaches in applying FHE to NN models. In this approach, distinct locally-optimized CCs are determined for each layer and used to generate FHE code tailored to the computation of each specific layer. Theoretically, this approach should promise optimal performance across the entire NN by (optimally) executing each layer over encrypted data with its dedicated CC. Evidently, however, the practical application of this approach in NNs comes at a cost. It requires additional cryptographic algorithms to switch CCs between layers, incurring non-negligible computational overhead [11–13]. As can be expected, in many cases, the CC switching overhead outweighs the performance benefits, consequently making this layer-wise optimization approach more expensive than the traditional one that maintains a single CC throughout the entire NN inference computation.

The purpose of this paper is to present our technique, called *LOHEN*, that aims to harness the performance benefits of layer-wise optimization while mitigating the extra overhead associated with CC switching. We have developed a cost estimation method that evaluates the computational expense of processing each layer with various FHE routines, and assessed the performance of LOHEN on multiple NNs. The details of LOHEN will be elaborated in the subsequent sections. To verify the effectiveness of our technique, we conducted experiments with two different settings. In the first one, our goal was using LOHEN to maximize the NN inference performance based on the layer-wise optimization approach. For this goal, we first implemented LOHEN using an state-of-the-art FHE scheme, *CKKS*. In the implementation, LOHEN first performs layer-wise optimization by determining locally-optimal CKKS CCs for each layer, and calculates the costs of CC switching between layers. Then, it modifies or merges the

CCs assigned to the layers in a way that minimizes the total computation time (= layer computation time + CC switching time) for the entire NN inference. In the experiment, we measured the performance of state-of-the-art techniques that allocates one good CC for the entire NN computation using CKKS, and compared these results with those achieved by LOHEN. The comparative analysis revealed that LOHEN outperforms them by a factor of up to $2.8\times$.

In the second setting of our experiment, we extend the application of layer-wise optimization to encompass not only performance but also accuracy. To elaborate, we hereby note that different FHE schemes have their own strengths and weaknesses in terms of accuracy and performance. For example, CKKS inherently does not support non-arithmetic functions [14]. Hence when researchers apply CKKS to a NN layer with non-arithmetic operations, they mostly resort to approximating these operations, inevitably inducing accuracy loss [4, 8]. On the other hand, TFHE, which we pick here for our experiment, can compute arbitrary functions (including non-arithmetic ones) using a specialized routine called programmable bootstrapping (PBS) [15, 16], offering precise computation for NN layers regardless of whether they involve arithmetic or non-arithmetic operations. Given this, if someone aims to exert NN inference over encrypted data with no loss in accuracy, a straightforward method would be replacing CKKS with TFHE for encrypting NN data. However, TFHE exhibits a significant performance degradation of several orders of magnitude compared to CKKS. Thus, this naive substitution of TFHE for CKKS would incur a considerable amount of performance degradation.

In our second experimental setting, our goal is to upgrade LOHEN such that we can perform NN computing without sacrificing accuracy, yet achieving the performance comparable to that of previous work which prioritizes performance over accuracy by approximating non-arithmetic operations whenever necessary. To realize our goal, we have modified the LOHEN implementation for our first experiment such that LOHEN sparingly use the TFHE scheme only for layers containing non-arithmetic operations and the faster CKKS scheme for all the rest. Unfortunately, due to the involvement of TFHE along with CKKS in NN inference, this basic *layer-wise application of multi-schemes* (CKKS+TFHE) still suffers from high computing time as compared to prior solutions that exclusively apply CKKS to the NN. Nevertheless, after employing various optimizations, which is detailed later, we managed to enhance performance significantly. Moreover, when integrated with LOHEN’s layer-wise optimization for CC selection used in the first experiment, we further excel the performance. We evaluated LOHEN in terms of accuracy and performance by comparing the results with those of the conventional work that only uses CKKS with one CC applied to the entire NN model. Whereas the compared baselines exhibit 2.7–4.6% accuracy loss, LOHEN with our layer-wise multi-schemes application achieves zero loss in accuracy,

Table 1: Notations

R_N	: polynomial ring of degree N
s, ct	: secret key & ciphertext encrypted with it
$\mathbf{m}, m(X)$	: message vector & plaintext polynomial
LWE_q^n	: LWE ciphertext (dimension n , modulus q)
$RLWE_{q,n}$	: $RLWE$ ciphertext (degree n , modulus q)
$\mathbf{P}, \lambda$	: data precision & security level
h_i, w_i, h_o, w_o	: height, weight of input/output tensor
c_i, c_o	: number of channels input/output tensor
f_h, f_w, st	: kernel size & stride of convolution

even attaining up to $1.42\times$ enhancement in performance.

Our two experimental settings demonstrate that when developers use FHE for privacy-preserving ML, LOHEN enables them to customize optimizations to adjust the desired levels of accuracy and performance of NN operations on encrypted data, catering to their specific needs. For instance, in the experiments, we see that if developers’ top priority is accuracy, LOHEN attains no or little loss in accuracy while yielding results comparable to prior works. Conversely, when performance is prioritized absolutely over accuracy, LOHEN achieves a significant enhancement in performance, multiple times better than previous works albeit with an accuracy loss akin to them. Furthermore, if developers go for outcomes that lie somewhere between these two extreme cases, LOHEN can deliver improved performance relative to the former and enhanced accuracy compared to the latter.

The rest of this paper is organized as follows. Section 2 introduces preliminaries on FHE and CC switching (§2). Section 3 presents our observations on the performance of NN computations using a single CC and layer-wise CCs (§3). Section 4 describes the design of LOHEN and its layer-wise optimization techniques. We first introduce a cryptographic gadget used for CC switching (§4.2) and describe our cost estimation analysis used to optimize the computation time of each layer (§4.5). In section 5, we then describe LOHEN integrated with multi-schemes (§5), which not only targets optimizing performance but also accuracy. Our evaluation of the two settings are presented in Section 6, where we compare with state-of-the-art baselines and present the impact of each design choices. Finally, related work and conclusion are presented in Section 7 and 8, respectively (§7, §8).

2 Background

This section describes the essential preliminaries of our work. We refer to the literature [14, 15] for cryptographic details. Table 1 lists the notations that we will use in this paper.

2.1 CKKS

CKKS [14] encodes and encrypts multiple fixed-point data elements into a single RLWE ciphertext, a process referred to as *packing*. Using the method, The arithmetics over CKKS operate in a SIMD-manner (*i.e.*, element-wise parallel computing).

Table 2: Latency of CKKS operations (GPU, $N = 2^{17}$, $\log q = 54$)

level (μs)	4	6	8	10	12	14
PMult	31	36	41	49	53	63
CMult	1004	1297	1624	2067	2462	2973
Rot($<<1$)	927	1198	1504	1925	2293	2777

Table 3: Latency of rotations on (GPU, $N = 2^{17}$, $lvl = 7$)

offset (μs)	1	2	3	4	5	6	7
Rot	960	971	1796	956	1782	1892	2831

For a power-of-two integer $N \geq 2$, we define a polynomial ring with modulus q as $R_{q,N} = \mathbb{Z}_q[X]/(X^N + 1) = R_N/qR_N$. The message vector $\mathbf{m} \in \mathbb{C}^{\frac{N}{2}}$ is encoded to a plaintext polynomial $m = \text{Ecd}(\mathbf{m}) \in R_N$. The RLWE encryption of a polynomial m using the secret key $s \in R_N$ is computed as $(b, a) = (m + e - a \cdot s, a) \in R_{q,N}^2$, where $a \in R_{q,N}$ and $e \in R_N$ is chosen from a uniform distribution and distribution of a small standard deviation, respectively. $\langle ct, s \rangle$ denotes the RLWE decryption which is computed as $(b, a) \cdot (1, s) = m + e$. Ciphertexts can be computed to each other, following pre-designed routines such as rotation and BTS. We describe the representative ones below :

Addition/Multiplication. The arithmetics between two ciphertexts ct_1 and ct_2 which encrypts the messages $\mathbf{m}_1$ and $\mathbf{m}_2$ results in a ciphertext that encrypts the element-wise arithmetic operation between $\mathbf{m}_1$ and $\mathbf{m}_2$. Table 2 depicts the latency of CKKS operations run on GPU. The latency of multiplication grows linearly with the level.

Rescale. After successive multiplications on a ciphertext, the scale of is doubled (*i.e.* Δ^2). To avoid the accumulated scale from exceeding its upper bound Q , we need to perform the routine called rescale that reduces the lvl .

Rotation. Rotation is a circular shift within the encrypted vector. Given an RLWE ciphertext ct that encrypts m , the rotation of ct by an offset k results in the encryption of m' where $\mathbf{m}' = \mathbf{m} \ll k$. Its cost increases along with the bit length of k , as shown in Table 3. This is because typical FHE libraries [17, 18] only generate rotation keys for offsets of power of 2, and use them sequentially to form other rotations.

Bootstrapping (BTS). When a ciphertext reaches a certain level that is unable to perform more multiplication, BTS should be performed to recover Q and lvl . Among many variants, we use the one that offers a modest precision [19], which requires a minimum of three lvl left in a ciphertext in order to initiate. BTS is generally known to take longer than a single multiplication at a scale of 2 to 3 orders of magnitude [20] (we describe its process in App. C).

2.2 FHE over the Torus (TFHE)

TFHE (also called CGGI) is a LWE/GSW-based FHE scheme. $LWE_q^n(\hat{m})$ denotes the LWE encryption of the message $\hat{m} \in \mathbb{Z}_q$ using the secret key $\hat{s} \in \mathbb{Z}_q^n$, computed as $(\hat{b}, \hat{a}) = (\hat{m} + \hat{e} - \hat{a}^\top \hat{s}, \hat{a}) \in \mathbb{Z}_q^{n+1}$ where $\hat{a} \in \mathbb{Z}_q^n$ and $\hat{e} \in \mathbb{Z}_q$ are randomly sam-

Table 4: CC factors

$\log q$	:	bitwidth of each modulus
q_0	:	base modulus
Q	:	ciphertext modulus at the highest level
L_v	:	max level of the ciphertext
N	:	degree of CKKS ciphertext

Table 5: The security requirements for $\lambda = 128$. For each N, Q should not be bigger than the presented number.

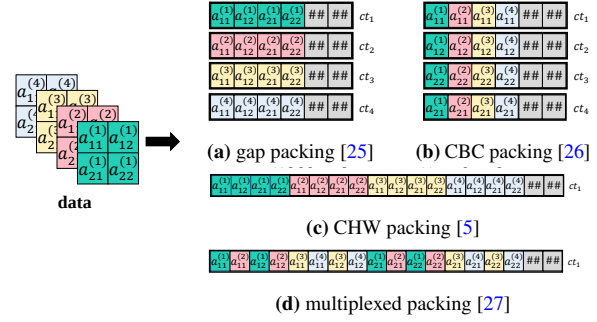
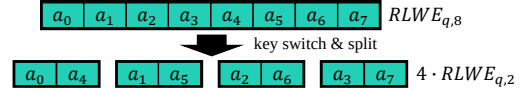
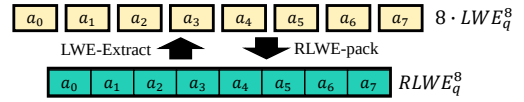
	N	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}
$\log Q$	27	54	109	218	438	881	

pled. $\langle \hat{ct}, \hat{s} \rangle$ denotes a decryption, which is $(\hat{b}, \hat{a}) \cdot (1, \hat{s}) = \hat{m} + \hat{e}$. The main feature that differs from CKKS is Programmable BTS (PBS) [15, 21], which allows to compute an arbitrary function, making TFHE adequate for non-arithmetic tasks [22–24]. Ironically, the critical downside of TFHE lies within PBS itself. It does not support packing, having comparatively lower BTS throughput compared to CKKS. Also, TFHE is unable to hold deep multiplicative depth [21].

2.3 Ciphertext Configuration (CC)

Ciphertexts can be configured based on various CC factors, such as the modulus chain and the degree of the ciphertext. The CC factors used in this paper are listed in Table 4. CC determines the security level and data precision, as well as overall performance of the FHE code. The polynomial degree N and Q determine the security level λ following the (R)LWE problem. Table 5 gives the maximum Q depending on N when s is sampled from ternary distribution. To ensure the desired data precision, the ciphertext should be carefully structured, selecting the appropriate CC factors such as base modulus q_0 and q . q is the significant figures bit length, which is the number of bits to store the encoded numbers. Upon decryption, the stored value is transitioned to q_0 , which has a longer length, added with the error. Typically, FHE libraries [17, 18] using the modest q_0 of 54 bits gives us around 32 bits of precision. Finally, performance usually depends on the ciphertext size (N and Q). N and Q affect the latency of operations over ciphertexts such as addition, multiplication and rotation, which usually have time complexities of $O(N)$ or $(O(N \log(N)))$.

CC Selection. We denote the decision-making during the pre-setting phase of using FHE as *CC selection*, where the ciphertext’s format is determined along with the vectorization of the encrypted data. The custom in FHE computing is to select one CC to be used as a global (uniform) configuration during the whole FHE computation. Researchers have been suggesting methods in selecting a global CC for general purpose computation [2, 5] or how to use a certain CC effectively [6, 20, 28–30], where most of them focused on the vectorization as well as scheduling certain FHE routines. Among them, the recent COYOTE [6] exhibits great performance in selecting the data layout for a given work-


Fig. 1: Example of packing methods for data $(h_i, w_i, c_i) = (2, 2, 4)$

Fig. 2: Example of Ringswitch to lower degree

Fig. 3: Example of LWE-Extract and RLWE-Pack.

load. Several works also put efforts into adopting the conventional method of selecting the global CC for NN computation. Specifically, they focused on designing efficient data packing methodologies and selecting the optimal CC accordingly. Numerous works [3, 8, 31, 32] introduced manual CC selection approaches retrofitted to NNs. Figure 1 depicts several packing methodologies from prior works. Among them, FHE-MP-CNN [8] is the state-of-the-art NN inference work using CKKS that shows the best balance between performance and accuracy by selecting a CC with the goal of maximizing the slot utilization (*i.e.* minimizing the number of ciphertexts), which we use as the baseline CC selection used throughout our design and evaluation of LOHEN. Hyphen [31] also introduces a method, which we will hereafter call *LoLat*, tailored to minimize the rotation and bootstrapping costs.

2.4 CC Swtiching

Fig. 2 depicts the former, where a ring with degree 8 is switched to a ring with degree 2. Our work follows Ringswitch explained in literatures [33, 34], of which the latencies are depicted in Table 6—note that the latency is related to the bigger N , whether it is the destination or the source.

Chilloti et.al. [15] demonstrated that it is possible to switch

Table 6: Latency of Ringswitch to/from n/N .

n/N (μs)	2^{14}	2^{15}	2^{16}
2^{14}	-	89	161
2^{15}	89	-	161
2^{16}	161	161	-

from one format to another, including LWE/RLWE conversions. Several works introduced new routines with different purposes [11–13, 34]. We present the three representative CC switching routines from the prior works. We also use them to devise LOHEN Cryptographic Repacking.

Ringswitch. Ringswitch [33, 34] switches the ciphertext to a different ring. Suppose we have large/small rings with degrees N/n , respectively. The conversion could be processed in both directions. Ringswitch from R_N to R_n requires key switching operations to split the ciphertext into N/n pieces. The opposite combines divided pieces of ciphertexts and switches the key in degree N . Whereas bigger N enables using bigger Q and bigger L_v , increasing N does not naturally provide this benefit—another operation, modRecover, which offers similar functionality as BTS as well as latency.

LWE-Extract. With respect to the coefficient-encoded RLWE ciphertext in $R_{q,n}^2$, one can extract the values of individual coefficients in its plaintext as n LWE ciphertexts in $\mathbb{Z}_q^{n+1}$. The function Extract^i [15] extracts the i -th coefficient m_i of the plaintext polynomial $m(X) \in R_{q,n}$, such that

$$m(X) = m_0 + m_1X + \dots + m_{n-1}X^{n-1}$$

RLWE-Pack. This operation [15] packs n LWE ciphertexts in dimension n into an RLWE ciphertext. This assigns the message m_i of the i -th LWE ciphertext to the i -th coefficient of plaintext $m(X)$ of a single RLWE ciphertext. It is recognized as an inverse routine of LWE-Extract, as depicted in Fig. 3. Note that weaving LWE-Extract and RLWE-Pack enables us to perform permutations instead of rotations, which is one of the key benefits of CC switching that we exploit.

2.5 NN Notations

Throughout our literature, we denote the inference tasks following the rules described below. The first letter represents the dataset (C, I for CIFAR-10, and ImageNet, respectively), and the second letter states the NN architecture (R, V, SQ, and MO for ResNet, and VGG, SqueezeNet, and MobileNet). The number appearing right after these two letters is the depth of the NN architecture if necessary. For example, CR20 stands for ResNet-20 using the CIFAR-10 dataset, and ISQ stands for SqueezeNet using the ImageNet dataset.

3 Applying FHE to Neural Networks

Each NN consists of distinct multiple layers computed sequentially. Every layer can be considered an independent computation workload assembled to form one complete NN. Notations for NN parameters are in Table 1. We present our observations made when applying a FHE scheme to NNs as below. We choose CKKS as the scheme.

Suboptimality of Globally Uniform CC. All prior works on running CNN over FHE use a single, globally uniform CC to

Table 7: FHE-MP-CNN evaluations using GPU.

seconds(%)	Rotation	Others	BTS	SUM
CR20	4.08 (20%)	4.69 (24%)	11.32 (56%)	20.10
IR18	266.40 (68%)	24.09 (6%)	101.91 (26%)	392.40

Table 8: Structural factors $w_o \times h_o, c_o$ in CR20 and IR18

Layer Group	Conv-1	Conv-2	Conv-3	Conv-4	Conv-5
CR20	32 × 32, 16	16 × 16, 32	16 × 16, 32	8 × 8, 64	8 × 8, 64
IR18	112 × 112, 64	56 × 56, 64	28 × 28, 128	14 × 14, 256	7 × 7, 512

compute NNs from the input through the output. This design decision is regarded as unavoidable because CC is determined when the input is encrypted. Unfortunately, using a single, globally uniform CC in all layers of an NN is suboptimal.

Fig. 4 depicts an example of two consecutive convolution functions with different hyperparameters, where encrypted convolutions are computed using the state-of-the-art multiplexed parallel packing [8], where CCs are selected accordingly. Note that Rotation takes much more time than PMult, dominating the overhead Table 2.

Since the cost of each HE operation in $N = 8$ is about half compared to that in $N = 16$ (i.e., different N induces different cost), the optimal CC N for CONV is 8. However, selecting a uniform CC with $N = 16$ is more beneficial at a global view encompassing both convolutions even if CONV runs over suboptimal N . Similarly, when running NNs over CKKS, each layer has a distinct level of parallelism determined by the NN architecture and the dimension of the input tensor. Accordingly, the optimal selection of the parameter N typically varies across different layers, making the use of globally uniform polynomial degree N , which is suboptimal. While this is considered an unavoidable constraint for the CKKS-applied NNs, we introduce our approach to another option in §4.

The Dominant Rotation Costs between Layers. During the inference of a NN, the resulting vector from one layer must always be restructured, i.e., permuted to be used as the input for the succeeding layer, in order to maximize the benefits of SIMD computation. In CKKS, this is done with the help of rotation operation. The overhead becomes much bigger for widely used NNs because they use longer vectors and more kernel operations. This cost, sometimes even dominates the end-to-end inference latency, especially for NNs with various dimensions. Table 7 shows the breakdown of latency when running 2 NN inferences using FHE-MP-CNN [35, 36], the state-of-the-art approach in running CKKS-applied CNNs, and Table 8 shows some of the structural factors of the two NNs. CR20 and IR18 have similar depths, but different input size, whereas the structural factors of CR20 only range to 64, whereas IR18 uses factors up to 512. Also, the dimension of output tensor ($w_o \times h_o \times c_o$) changes vigorously in IR18 while in CR20, the changes are rather calm (e.g., the first three layer groups only differ in c_o). This is why the latency of rotation becomes more dominant for IR18 compared with CR20.

Layer-Wise CC Induces More Costs. Note from Fig. 4

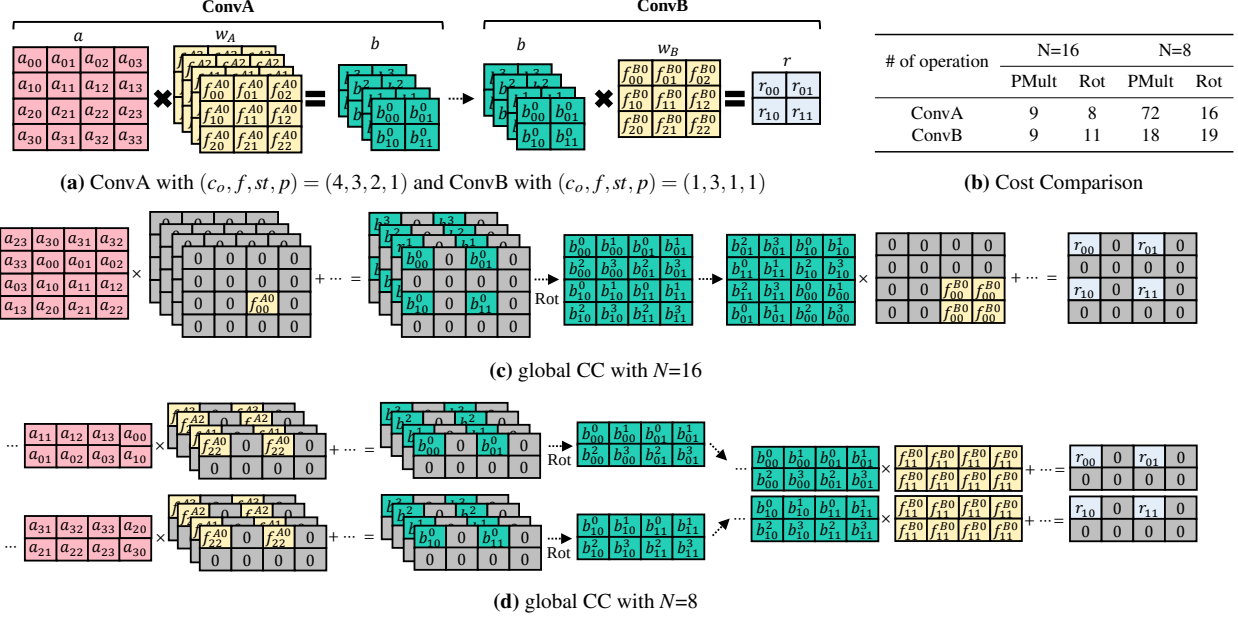


Fig. 4: Comparison of global CC for Consecutive Convolution with different N

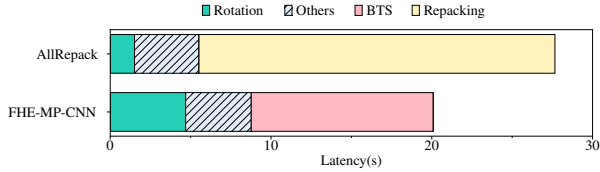


Fig. 5: Latency of CR20 using Layer-wise CC and FHE-MP-CNN

that the computation cost of the two convolutions using local CCs gives the best performance for their computation, taking 8 rotations for CONVA with $N = 16$ and 19 rotations CONVB with $N = 8$ each. There are two aspects of the benefit of adopting local CCs. First, locally optimal CC reduces the intra overhead of each function – the number of operations is reduced as well as their individual cost. Second, it is possible to perform arbitrary permutation during CC switching as described in §2.4 and avoid using rotations between layers. Despite the benefits, this layer-wise CC allocation does not always outperform the use of global CC and rotation-based permutation, due to the excessive overhead required for CC switching. Fig. 5 shows the latency breakdown of CR20 inference before and after applying local optimal CCs to all layers by applying "AllRepack" referring to when we apply an efficient CC switching we devised and used in our work, called "LOHEN cryptographic repacking" which will be introduced in the later section. Allocating to each layer its local optimal CC reduces the costs of FHE operations and avoids certain portions of rotations – it also replaces BTS. However, even with the help of more optimal CC and reduced overhead for permutations, the end-to-end latency becomes longer, suggesting that CC switching improves the inference latency only when used selectively. To address this issue, we

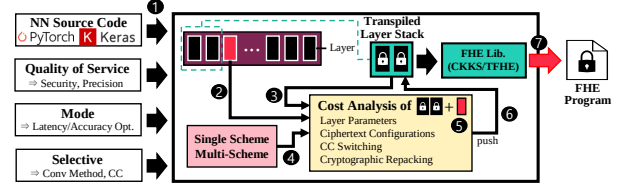


Fig. 6: LOHEN Overview

design LOHEN, as will be described below.

4 LOHEN for Performance

We address the challenge introduced in §3 by devising a cryptographic gadget for CC switching, which we call *LOHEN cryptographic repacking (LCR)*, and implement an automated method to determine if each permutation between layers should be replaced with LCR as well as selecting a CC for each layer. We have identified the feasibility of constructing a cryptographic gadget that efficiently supplements the rotation-based permutation mechanism and enables the modification of CC when running CKKS-applied NNs. LOHEN automatically derives the estimated latency cost of each operation at each point in the NN without running it to avoid the potentially excessive cost of dynamic analysis. It then uses the cost estimation to determine optimal CC mapping at each point in an iterative way, as we detail in §4.5.

4.1 LOHEN Overview

LOHEN uses per-layer CC and LCR to automatically produce a latency-optimized executable representation of NN

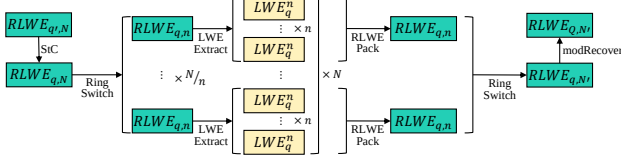


Fig. 7: Illustration of LCR

inference over encrypted data from a description of a NN computation in widely used NN frameworks (e.g., PyTorch). When requested by the developer, it is also capable of balancing between latency and accuracy by selectively adopting TFHE-based non-linear layers. Fig. 6 gives an overview of LOHEN. LOHEN takes a description of NN computation (i.e., the source code) written in machine learning frameworks such as PyTorch or Keras as the input. The source code is first transformed into an intermediate format that summarizes each layer’s characteristics, such as kinds of operations or convolution filter types (❶). From this layer-wise information, LOHEN uses its analyzer to estimate the cost of each building block and uses the result to select and schedule the FHE operations for each layer, one by one, from the first layer.

Fig. 6 illustrates the behavior of LOHEN when it works on the third layer. To analyze, estimate, select, and schedule the FHE operations for the third layer, LOHEN takes the intermediate result that contains the results for the first two layers (❷) and the composition of the layer (❸) as the input and analyzes the cost (❹). Through this analysis, LOHEN determines the optimal choice of the CC and pushes to the intermediate results (❺) for the following steps. The candidates of the operations considered in these steps include LCR §4.2. The TFHE-based non-linear layers §5 are also considered if the user enables it (❻). Once all layers are analyzed, LOHEN emits a FHE program corresponding to the computation (❼). During these steps, LOHENs cost analyzer automatically considers various constraints such as the target security level (i.e., λ), precision (e.g., 32-bit), or the user’s desired optimization priority (i.e., latency or accuracy). §4.5 describes more details about the cost analyzer, and §4.3 explains the algorithm.

4.2 LOHEN Cryptographic Repacking (LCR)

Fig. 7 illustrates LCR, designed for LOHEN to enable CC switching and to replace the rotation-based permutations between layers. The gadget is built with the primitive operations for RLWE and LWE ciphertexts that prior works have designed [15, 33, 34]. Taking an RLWE ciphertext as an input, the repacking gadget performs SlotCoeff (StC) [37], which changes a slot-represented ciphertext into coefficient form. Followed by this StC, LOHEN should run the LWE-Extract and RLWE-Pack for LCR.

A key design decision to enhance performance involves the use of Ringswitch immediately after StC because the subsequent operations exhibit reduced latency when applied to

Table 9: BTS and LCR latency on GPU (μ s).

logN	15	16	17
BTS	33,766	96,964	371,541
Repack	47,777	111,012	402,749

ciphertexts of lower degrees. Specifically, LCR gadget splits a RLWE ciphertext with degree N into multiple RLWE ciphertexts with the degree $n = 2^{11}$, which is almost always lower than the degree N of the input ciphertext. We choose 2^{11} as the target degree because it is the smallest degree that provides non-zero multiplicative depth under the 128-bit security (i.e., $\lambda = 128$) when $q = 54$, both of which are the de-facto standard choice of parameters. Note that the optimal target degree may vary when these two changes. LOHEN runs LWE-Extract and RLWE-Pack in sequence to extract the LWE from RLWE ciphertexts and, and then pack the latter into one or more RLWE ciphertexts back. The last step of LCR is the ModRecover that recovers the level (lv) of the target ciphertexts, which has the same functionality as BTS.

CC Switching by LCR. LOHEN capitalizes on three fundamental aspects of LCR strategy. First, the aforementioned design choice enables LOHEN to switch the ciphertext degree efficiently when it performs LCR. LCR already performs Ringswitch while reconstructing the RLWE ciphertext, and LOHEN can choose to use a degree that is not the same as that of the original ciphertext if its cost analyzer reports that using a new degree delivers lower latency. Second, LCR also obviates the rotations between layers of a CNN because LOHEN can permute the vector elements of ciphertexts during RLWE-Pack. Third, LCR also involves the role of BTS because it recovers lv . LCR alone is not an efficient replacement for BTS, as Table 9 shows. However, this aspect makes the cost of adopting LCR lower, along with the second aspect, when it also replaces BTS while performing CC switching.

4.3 Layer-wise CC and Operation Selection

As explained earlier, LOHEN determines the optimal CC and the corresponding operations for each layer using the results from the earlier ones. In this context, Algo. 1 shows how LOHEN works for the k -th layer, denoted as l_k in the algorithm. Inputs to this algorithm include the Closed Layer Stack (CLS) and Active Layer Stack (ALS). CLS is a stack of layers for which LOHEN already determined the CC, and ALS contains the ones that LOHEN may still change their CC. After executing this step, LOHEN may push l_k to the ALS and move on to the next layer (l_{k+1}) or decide to consider l_k again. If the latter happens, it pushes all elements from ALS to the CLS and empties the ALS. Both ALS and CLS are empty when LOHEN handles the first layer, and CLS contains all layers after LOHEN finishes handling the last layer.

The algorithm begins by computing the basis latency cost of a newly incoming layer, l_k , denoted as $Cost(l_k)$ (line 4). It

Algorithm 1: Layer-wise cost estimation analysis.

Input: *CLS*: Closed Layer Stack, *ALS*: Active Layer Stack,
 l_k : current layer

Output: *ALS*, *CLS*, l_{next}

```

1 Init
2  $Cost_{static}(l_k, N) = A_k \cdot BC_A \cdot k_A(N) + P_k \cdot BC_P \cdot k_P(N)$ 
3  $+ C_k \cdot BC_C \cdot k_C(N) + R_k \cdot BC_R \cdot k_R(N)$ 
4  $Cost(l_k) = (Cost_{static}(l_k) + Cost_{dyn}(l_k))$ 
5 if ALS.getRemainlvlSlot( $l_k$ ) == Minlvl(ALS,  $l_k$ ) then
    // need to repack or enlarge N
6     if EnlargeCost( $l_k$ ) < RepackCost( $l_k$ ) then
7         ALS.expand( $N$ ,  $N + 1$ ); ALS.push( $l_k$ );
8          $Cost(l_k) \times = k(N + 1) / k(N)$ ;
9         ALS.cost  $\times = (1 + 1 / \text{ALS.lvl})$ ;
10        ALS.cost += Cost( $l_k$ );  $l_{next} = l_{k+1}$ ;
11    else
12         $l_k \rightarrow PERM = 0$ ; ALS.cost += RepackCost( $l_k$ )
13        ALS.push(Repack);  $l_{next} = l_k$ 
14        CLS.push(ALS); ALS  $\leftarrow$  new ALS;
15 else
    // push L to ALS without repack
16    if  $Cost_{dyn}(l_k) + \text{ALS.cost} \times (1 / \text{ALS.lvl}) <$ 
        RepackCost( $l_k$ ) then
17        ALS.cost  $\times = (1 + 1 / \text{ALS.lvl})$ ;
18        ALS.cost += Cost( $l_k$ ); ALS.push( $l_k$ );  $l_{next} = l_{k+1}$ ;
    // push repack to ALS and close the stack
19    else
20         $Cost_{dyn}(l_k) = 0$ ; ALS.cost += RepackCost( $l_k$ );
21        ALS.push(Repack);  $l_{next} = l_k$ 
22        CLS.push(ALS); ALS  $\leftarrow$  new ALS;

```

subsequently compares the required multiplicative depth, or *lvl*, when this l_k is pushed to ALS and the remaining *lvl* considering the currently chosen candidate CC for ALS (line 5). If ALS will have sufficient budget for multiplication (*i.e.*, *lvl*) left even after having l_k , the algorithm moves to compare the cost of LCR and rotation-based permutation (line 16). Specifically, LOHEN pushes l_k to ALS and moves on to the next layer if LCR is more expensive. If LCR is found to be more efficient, it pushes LCR to ALS and revisits l_k . After pushing LCR to ALS, LOHEN closes ALS and appends it to CLS so that it can start from a fresh, empty ALS because the added LCR operation will fresh out the CC and recover *lvl*.

If the addition of l_k to ALS consumes all remaining *lvl*, *i.e.*, requires the execution of BTS right after l_k , LOHEN considers the increase of degree (N) and LCR as two possible options. Specifically, LOHEN compares the additional latency cost of ALS if the degree is enlarged for the entire ALS, including l_k , with the cost of LCR (line 6). If LCR is found to be more efficient, LOHEN pushes repack to ALS, pushes it to CLS, and starts again for l_k with fresh and empty CLS (line 12–line 14). Otherwise, it changes the CC of ALS, *i.e.*, enlarges the degree, and pushes l_k to ALS. Note that this can be done because enlarging N increases the *lvl* budget of the ALS. Fig. 8 depicts an example of how our cost analysis works

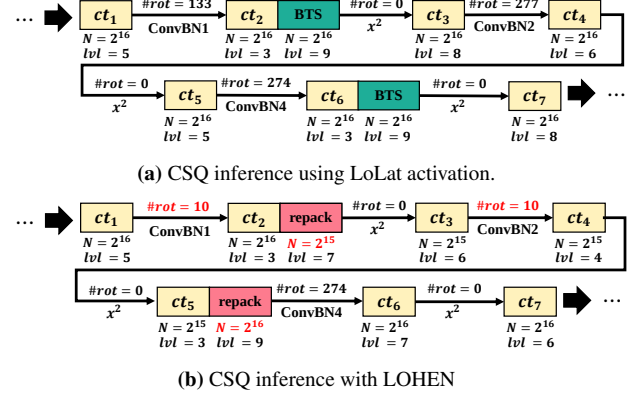


Fig. 8: Example of our cost analysis during CSQ Inference.

for the CSQ workload. Using the LCR, LOHEN reduces the number of rotations in ConvBN1 and ConvBN2 to 10 from 133 and 277, respectively. It also reduces the ring size of ct_3 , ct_4 , and ct_5 to $N = 2^{15}$.

4.4 User-specified Constraints

LOHEN considers user-specified security and performance constraints when it computes the latency costs and chooses CC. For example, they may require LOHEN to follow a particular optimization technique to perform convolutions using CKKS (§2 presents the examples) or instruct how it would approximate the non-linear layers. The user can also pre-define security level (*e.g.*, $\lambda = 128$) and data precision, which affects the quality of the service, such as the inference latency or accuracy. In detail, these constraints affect several CC factors or narrow down the options available for LOHEN. If not others specified, LOHEN uses a constraint, where $\lambda = 128$ (*i.e.*, security level of 128-bit) and 32-bit precision (corresponding to $q_0 = 54$). For the others, it takes the configuration used by FHE-MP-CNN [8]. In all descriptions of LOHEN and experiments, we use this default constraint.

4.5 Estimating Layer- and ALS-wise Cost

The cost analyzer is a key building block that LOHEN relies on. It estimates the cost in terms of inference latency for either one layer (*e.g.*, l_k) or a stack of layers (*e.g.*, ALS).

Estimating for One Layer. To estimate the cost of a layer l_k , we use the number of primitive operations composing this layer to obtain the basis cost and multiply it with the degree-dependent factor. We first estimate the basis cost by combining the static and dynamic portion of the cost. The static portion comes from Add, PMult, CMult, and some Rotation operations because these are executed regardless of how the layer is combined with the others. The dynamic portion comes from the cost of permutation, which is also composed of rotations. We consider the cost of permutation as dynamic

because its cost may vary depending on the way this layer is composed – their numbers are determined by the approach we adopt (FHE-MP-CNN in our default settings, illustrated in App. D) LOHEN estimates the static cost using the number of static operations and their individual costs, which are empirically profiled. It estimates the dynamic cost ($Cost_{dyn}(l_k)$) using the number of rotations required and BC_R , which is reset to 0 if the layer is to be preceded by the LCR. (line 20). Accordingly, LOHEN estimates the cost for each degree N using the asymptotic time complexity of each operation.

$$Cost_{static}(l_k, N) = A_k \cdot BC_A \cdot k_A(N) + P_k \cdot BC_P \cdot k_P(N) \\ + C_k \cdot BC_C \cdot k_C(N) + R_k \cdot BC_R \cdot k_R(N)$$

A_k, P_k, C_k , and R_k are the number of each operation in l_k and BC_A, BC_P, BC_C , and BC_R are their costs. N is the degree of the ciphertext, and k_A, k_P, k_C, k_R are the degree factors that have the following form, considering the time complexity.

$$k_A(N) = k_P(N) = \frac{N}{2^{11}} \quad k_C(N) = k_R(N) = \frac{N \log N}{2^{11} \cdot 11}$$

5 LOHEN for Accuracy and Performance

As mentioned earlier, LOHEN can be instructed to selectively use TFHE-based non-arithmetic layers to improve the accuracy, similarly to the prior works [11–13], possibly at the cost of additional inference latency. LCR can be extended to perform TFHE-based non-arithmetic computations. By using extended LCR, LOHEN can reduce the accuracy loss owing to using approximated non-arithmetic layers using CKKS, which is known to negatively impact the inference accuracy [8, 38].

Utilizing the additional TFHE-based non-linear layers expands the possible design space, which requires again an automated approach to determine where to use the layers and how to set each layer’s CC. The TFHE layers bring improved accuracy but are slower than their counterpart using CKKS [11, 39]. Replacing approximated non-linear layers with TFHE-based ones is, therefore, an act of paying the latency for accuracy, but it also introduces chances to switch CC. LOHEN also considers this additional design space by considering the TFHE-based layers and their LCR together to produce FHE programs fulfilling a user-instructed goal. We provide more details on the possible goals and how LOHEN achieves them in the following section.

5.1 Selective TFHE Application

A user can instruct LOHEN to utilize the TFHE layers in the following three ways. We discuss the difference between the prior works and LOHEN’s use of TFHE layers in §7.

Preliminary Optimizations. The extended LCR gadget performs computation over TFHE while the ciphertexts are in the form of LWE ciphertext. We first describe a straight-forward optimization that can easily be applied to our extended LCR

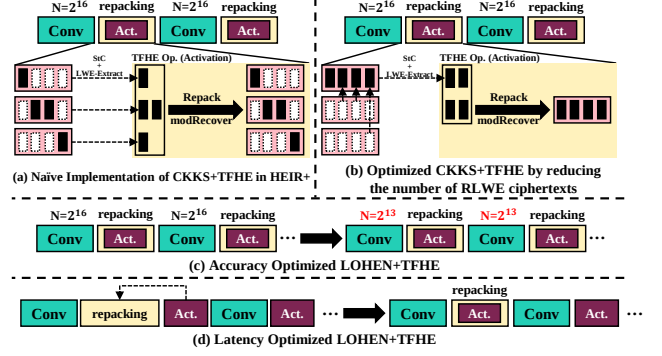


Fig. 9: Layer-wise Multi-Scheme Approaches

as well as previous works using multi-schemes [13, 39]. Instead of performing LCR (or scheme conversions for the prior works) on a ciphertext basis, it is possible to combine sparse ciphertexts into dense ones at the beginning of the gadget and pack them into a single RLWE ciphertext afterward. This optimization reduces the number of modRecover operations, whose latency takes a large portion of the gadget’s latency. Fig. 9-(a) and (b) highlight the effect where the number of modRecover is reduced from 3 to 1. The design also helps gather the scattered values in one ciphertext before engaging in LCR, reducing the amount of LWE-Extract and other components. Note that we *do not* consider this as part of LOHEN’s acceleration but rather a way to enhance the multi-scheme baseline’s performance we use in our evaluation in their favor.

Goal 1: Accuracy-oriented. If this goal is chosen, LOHEN uses TFHE layers for all non-arithmetic layers to compute the best possible precision and performs its layer-wise selection for the remaining layers. Fig. 9-(c) depicts an example running LOHEN in the accuracy-oriented mode. The two activation layers are replaced with the TFHE layers, which also perform the repacking, enabling LOHEN to switch CC at those points. LOHEN applies Algo. 1 to each of the separated groups of arithmetic layers (single convolutions in the example). As a result, each convolutions are set with $N = 2^{13}$, the smallest N having $lv = 4$, because we need it to have $lv = 3$ to initiate the subsequent TFHE layers with repacking.

Goal 2: Latency-oriented. For this goal, LOHEN employs TFHE layers only if its latency impact is expected to be small. To this end, it first inserts repacking as it would without considering the TFHE layers. From the result, only the non-arithmetic layers adjacent to the inserted repacking are considered to be accuracy improvement opportunities and replaced with the TFHE-based non-arithmetic layers. This is because the adjacent activation layer can be merged with the repacking, as we only need to add the TFHE computation inside the already existing repacking (*i.e.*, inducing no other extra cost for LWE-Extract, RLWE-Pack or modRecover). This approach improves the model accuracy when compared to the result of LOHEN without TFHE layers, but the result becomes slower because the non-arithmetic layers over TFHE

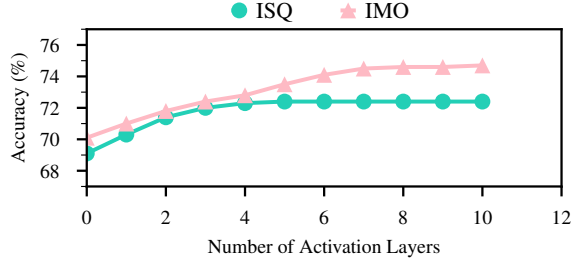


Fig. 10: Impact of the number of activation layers run over TFHE to accuracy over ImageNet inferences.

Table 10: Latency of LOHEN according to the number of activation layers computed with TFHE.

# layers in TFHE		0	5	10	...	ALL	Baseline (w/o. LOHEN)
ISQ	Latency(s)	1008	1297	1502	...	1690	2093
	Acc. Loss	-3.3	0	0	...	0	-3.3
IMO	Latency(s)	1016	1392	1573	...	2056	2923
	Acc. Loss	-4.6	-1.2	0	0	0	-4.6

are slower than those over CKKS. Note that the result of this goal produces a faster but less accurate model because not all non-arithmetic layers are replaced with TFHE. Fig. 9-(d) depicts an exemplary result of this approach, assuming that LOHEN placed a repacking after the first convolution layer.

Goal 3: Balanced. LOHEN can also aim somewhere between the two goals with the help of profiling. To balance the accuracy and latency, LOHEN profiles the model accuracy when replacing the first k non-arithmetic layers by changing k . It speculates the model’s accuracy for each k by measuring the accuracy with 1000 queries; its results when LOHEN runs for ISQ and IMO are shown in Fig. 10. LOHEN then chooses the smallest value of k that gives speculated accuracy the same as the encrypted model. Fundamentally, this balanced goal has the best-effort nature and may result in replacing all non-arithmetic layers with the TFHE ones. Nevertheless, we could obtain non-trivial results for the two workloads, which are 5 and 10, as Table 10 shows. We use these results for the evaluation later. Note again that the result of this balanced goal is not guaranteed to be different from the accuracy goal, and we decided to replace the non-arithmetic layers appearing earlier, referring to prior work on the relationship between the noisy layers and model accuracy [40].

6 Evaluation

We answer the following about the effectiveness of LOHEN.

- Does LOHEN improve the performance when applied to the state-of-the-art CNN over CKKS? (§6.1.1, §6.1.3)
- Does LOHEN also improve the performance of the latency-optimized variant? (§6.1.2, §6.1.3)

Table 11: The NNs and datasets used in our evaluation.

Name	Dataset	NN Architecture	Reference
CR20	CIFAR-10	ResNet-20	[41]
CV16	CIFAR-10	VGG-16	[42]
CSQ	CIFAR-10	SqueezeNet	[43, 44]
IR18	ImageNet	ResNet-18	[41]
ISQ	ImageNet	SqueezeNet	[44]
IMO	ImageNet	MobileNetv2	[45]

- Can LOHEN incorporate a TFHE-based non-linear layer for balancing the latency and NN accuracy? (§6.2)

Datasets and NNs. As the workloads for evaluation, we use the combinations of four widely used NNs (ResNet, VGG, squeezeNet, and mobilenet-v2) and two widely used data sets (CIFAR-10 and ImageNet). Table 11 presents the details of the combinations and their names that we use.

Accuracy Measurement. Unless otherwise specified, we use 1,000 queries from the validation set to obtain all accuracy numbers we report in this paper, and the accuracy refers to Top-1 accuracy. Note that this is a widely adopted method to measure the CNN inference accuracy.

Experimental Environment. We run all experiments using a machine that has two Intel Xeon Gold 6326 CPUs, 1TB of main memory, and one NVIDIA RTX 3090 GPU with 24GB of memory. We also used unified virtual memory (UVM). As the underlying FHE library, we used a customized version of the GPU library called Liberate-FHE [46]. We chose Liberate-FHE instead of SEAL, which FHE-MP-CNN [8] used for several reasons. First, SEAL does not natively support BTS. The SEAL variant that the authors of FHE-MP-CNN [8, 47] implemented for their work exhibits sluggish BTS compared to other implementations [17]. Also, Liberate-FHE is fully programmable for GPU. We make sure to compare ours with FHE-MP-CNN fairly by running it using our version of Liberate-FHE and comparing LOHEN against this result—the end-to-end latency of FHE-MP-CNN is significantly reduced, making it a more appropriate baseline for our study. We provide further details about differences in experimental environments in App. B. We also customized Liberate-FHE to support the FHE operations introduced in §2 in order to support full functionalities of CKKS efficiently using GPU. We modify this to enable RingSwitch, which changes the ring size, following the way introduced in a prior work [33]. We bind Liberate-FHE with NuFHE [48], a TFHE library for GPU.

6.1 LOHEN for Performance

6.1.1 Effectiveness on FHE-MP-CNN

FHE-MP-CNN. We evaluate the effectiveness of LOHEN when it is applied to FHE-MP-CNN [8] because it is considered as the state-of-the-art in encrypted CNN inference over CKKS, balancing the accuracy and latency by using its unique

Table 12: End-to-end latency of LOHEN applied to FHE-MP-CNN. AllRepack. replaces all rotation-based permutations with LCR

Workload	FHE-MP-CNN	AllRepack.	LOHEN
CR20	20.10s	29.35s (1.46 $\times$)	18.68s (0.93 $\times$)
CV16	72.60s	82.04s (1.13 $\times$)	46.65s (0.64 $\times$)
CSQ	44.42s	51.53s (1.16 $\times$)	32.75s (0.74 $\times$)
IR18	392.40s	302.15s (0.77 $\times$)	194.60s (0.50 $\times$)
ISQ	2092.62s	1757.80s (0.84 $\times$)	1007.96s (0.48 $\times$)
IMO	2922.78s	2279.77s (0.78 $\times$)	1016.17s (0.35 $\times$)

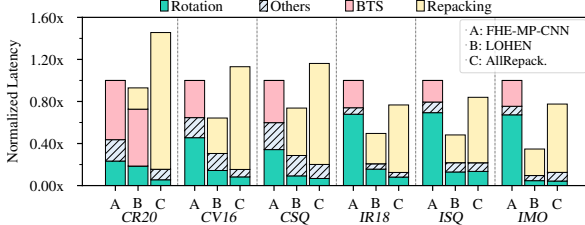


Fig. 11: Latency Breakdown of LOHEN(B) and AllRepack.(C) applied to FHE-MP-CNN(A) (Normalized to FHE-MP-CNN).

Table 13: The accuracy of workloads when running over plaintext space with different activation functions.

Workload	ReLU	x^2 (Used For LoLat)
CR20	91.8%	90.1% (-1.7%, 0.98 $\times$)
CV16	94.4%	92.8% (-1.6%, 0.98 $\times$)
CSQ	89.4%	88.1% (-1.3%, 0.99 $\times$)
IR18	70.2%	65.6% (-4.6%, 0.93 $\times$)
ISQ	72.4%	66.2% (-6.2%, 0.91 $\times$)
IMO	74.7%	67.4% (-7.3%, 0.90 $\times$)

multiplexed packing and high-precision ReLU approximation, enabling to use pre-trained models [49].

End-to-end Inference Latency. Table 12 shows the effectiveness of LOHEN in reducing the end-to-end inference latency. As the figure shows, LOHEN reduces the latency by 7–65%, which is equivalent to 1.08–2.88 $\times$ speedup when applied to FHE-MP-CNN, the state-of-the-art in inference over CKKS. It is also noteworthy that naïvely LCR at all layers, which we refer to as *AllRepack.* may not outperform the state-of-the-art and always have longer latency than LOHEN. AllRepack. increases the latency by 38% in the worst case.

Latency Breakdown. Fig. 11 depicts the breakdown of the normalized latency when we use LOHEN (B) and AllRepack. (C), when compared to the baseline FHE-MP-CNN (A). For all workloads, LOHEN and AllRepack. introduces the latency originating from LCR while reducing the rotation and BTS latency. AllRepack.’s inefficiency comes from the excessive latency of LCR, which is longer than the rotation and BTS latencies in CR20, CV16, and CSQ. On the contrary, LOHEN-introduced LCR latency is always shorter than the reduced rotation and BTS latencies, resulting in latency improvement.

6.1.2 Effectiveness on LoLat

LoLat. To better understand the potential effectiveness of

Table 14: End-to-end latency of LOHEN applied to LoLat AllRepack. replaces all rotation-based permutations with LCR.

Workload	LoLat	AllRepack.	LOHEN
CR20	7.23s	21.40s (2.96 $\times$)	5.71s (0.79 $\times$)
CV16	22.38s	39.84s (1.78 $\times$)	13.43s (0.60 $\times$)
CSQ	18.85s	45.05s (2.39 $\times$)	14.95s (0.79 $\times$)
IR18	85.77s	261.60s (3.05 $\times$)	76.34s (0.89 $\times$)
ISQ	931.03s	1396.55s (1.50 $\times$)	363.10s (0.39 $\times$)
IMO	1230.70s	2006.04s (1.63 $\times$)	627.66s (0.51 $\times$)

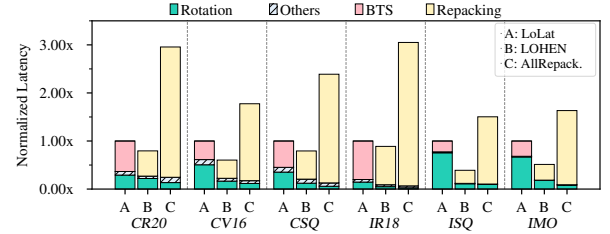


Fig. 12: Latency Breakdown of LOHEN and AllRepack. applied to LoLat activation. (Normalized to the baseline).

LOHEN on corner cases, we evaluate its effectiveness when applied to encrypted neural networks optimized for low inference latency. Specifically, this *LoLat* packs the inputs differently to reduce the number of rotations [31], and uses a simple square operation, x^2 [3, 9], as the activation function to reduce the number of multiplication, resulting in shorter BTS latency. Note that *LoLat* requires re-training the model, resulting in lower accuracy than those using ReLU – Table 13 shows that the use of the square for activation affects the inference accuracy negatively. This is the case where LOHEN is expected to be the least beneficial in improving the latency because LOHEN essentially uses the costs of rotation and BTS as the optimization opportunities.

End-to-end Inference Latency. Table 14 shows the end-to-end inference latency of LOHEN when compared to the unmodified LoLat baseline and the one with AllRepack. The reduced rotation latency makes the AllRepack much less effective, increasing the end-to-end latency by three times at most. In contrast, LOHEN automatically adapts to the situation and finds optimal replacements, showing lower latency in all workloads. Overall, LOHEN reduces the latency by 11–61%, corresponding to 1.12–2.56 $\times$ speedup.

Latency Breakdown. Fig. 12 depicts the latency breakdown of the original LoLat, LOHEN, and AllRepack. The design changes for latency reduction affect all three primary sources of latency, which are rotation, BTS, and ciphertext multiplication. This results in the difference in the relative latency of LoLat’s three components when compared to the FHE-MP-CNN. Note that despite the difference, the total rotation latency of LoLat is always lower than that of FHE-MP-CNN. Despite this, LOHEN uses the rotation and BTS latencies as the budget for latency reduction by selectively replacing them with LCR. AllRepack. suffers even more for LoLat because the total latency of rotation, which is what AllRepack. eliminates, is much shorter than that of FHE-MP-CNN while

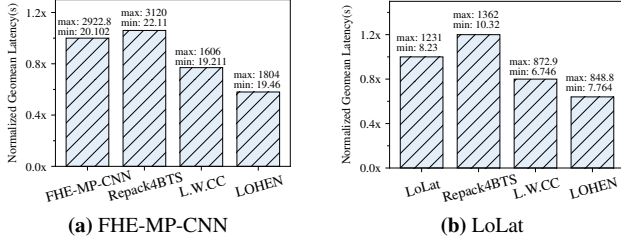


Fig. 13: Impact of CC and LCR

inducing a similar amount of LCR.

6.1.3 Impact of Design Choices

Fig. 13 shows the impact of individual design choices contributing to LOHEN’s performance improvement when running the six workloads we use throughout the evaluation. Due to space limitations, we only report the normalized latencies’ geometric mean, with the ranges annotated above each bar. The first bars give the reference point showing FHE-MP-CNN and LoLat results without any LOHEN components. To these, we first apply LCR only as a replacement for BTS, depicted as Repack4BTS. The result suggests that LCR is not an attractive gadget if it is used only to replace BTS, as it increases the latency by 6% and 20%. On top of this, we let LOHEN select the CCs of each set of layers separated by LCR, denoted as L.W.CC. As expected, LCR reveals its value when we take more of its potential by just picking CCs for each layer, resulting in 23% and 20% reduced inference latency, which are equivalent to $1.24\times$ and $1.33\times$ speedup. Capitalizing over another potential permutation while LCR further boosts the performance by reducing the latency by 36% and 42%, which are equivalent to $1.56\times$ and $1.72\times$ speedups. In summary, this study shows that LCR is beneficial only when used for more than a replacement of BTS.

6.2 LOHEN for Performance and Accuracy

Comparison Targets. We evaluate the effectiveness of LOHEN using TFHE-based non-linear layers selectively by comparing it with six alternatives in three categories. In the first category are TFHE-only alternatives, TFHE and SHE [50]. TFHE runs encrypted NN without any approximation, and SHE approximates multiplication with bit-shifts for performance. The second category is the CKKS-only ones, FHE-MP-CNN and LOHEN. The third category is the latest work that uses multi-schemes. HEIR [13] is the unmodified version of the latest work, and HEIR+Opt is a version that incorporates a mechanism inspired by LCR for lower latency. **Remark.** We note that this optimization is rather straightforward, for which reason we consider this HEIR+Opt variant as our comparison baseline in favor of HEIR. Also, note that LOHEN can be driven to prioritize either accuracy or latency, as we mentioned earlier.

Results. Table 15 shows accuracy and inference latency

Table 15: Latency and Accuracy of FHE-MP-NN, LOHEN, and MS-LOHEN when running ISQ and IMO workloads.

Workload		ISQ (72.4%)		IMO (74.7%)	
		Latency (sec.)	Accuracy (%)	Latency (sec.)	Accuracy (%)
TFHE	TFHE†	0.2M s (96.1 $\times$)	72.4% (-0.0%)	0.3M s (97.8 $\times$)	74.7% (-0.0%)
	SHE [50]†	0.2M s (8.28 $\times$)	69.4% (-3.0%)	0.3M s (10.6 $\times$)	70.6% (-4.1%)
CKKS	FHE-MP-NN	2093s (1.00 $\times$)	69.1% (-3.3%)	2923s (1.00 $\times$)	70.1% (-4.6%)
	LOHEN	1008s (0.48 $\times$)	69.1% (-3.3%)	1016s (0.35 $\times$)	70.1% (-4.6%)
Multi-Scheme	HEIR [13]	3006s (1.44 $\times$)	72.4% (-0.0%)	4481s (1.53 $\times$)	74.7% (-0.0%)
	HEIR+Opt.	2271s (1.09 $\times$)	72.4% (-0.0%)	3265s (1.12 $\times$)	74.7% (-0.0%)
	MS-LOHEN-A	1690s (0.81 $\times$)	72.4% (-0.0%)	2056s (0.70 $\times$)	74.7% (-0.0%)
	MS-LOHEN-L	1208s (0.58 $\times$)	71.2% (-1.2%)	1376s (0.47 $\times$)	71.9% (-2.8%)
	MS-LOHEN-B	1297s (0.62 $\times$)	72.4% (-0.0%)	1573s (0.54 $\times$)	74.7% (-0.0%)

M states for Million / †-projection of the first 2 layers / HEIR+Opt.: HEIR with opt. TFHE. MS-LOHEN-A / L: multi-scheme LOHEN prioritizing accuracy / latency. MS-LOHEN-B: multi-scheme LOHEN with profile-guided balancing.

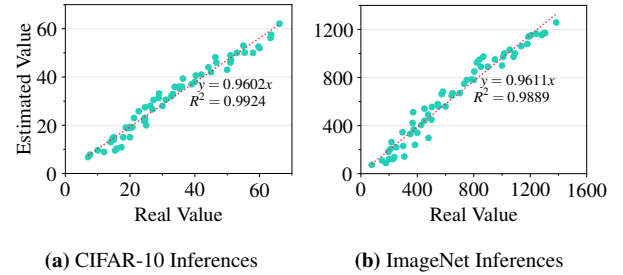


Fig. 14: Estimation error analysis of LOHEN.

when we run LOHEN using TFHE-based non-linear layers selectively when compared to the six possible alternatives. We only use the two workloads using ImageNet as the data set because the accuracy loss of FHE-MP-CNN for the others is much lower. The results show that MS-LOHEN achieves its goal. MS-LOHEN-A’s inference latency is 19% and 30% lower than FHE-MP-CNN, which is lower than all alternatives except for LOHEN and MS-LOHEN-L. It is noteworthy that MS-LOHEN-A achieves this latency without any accuracy loss, outperforming the multi-scheme baseline of HEIR+Opt., with a speedup of $1.34\times$ and $1.59\times$.

MS-LOHEN-L, on the other hand, trades latency with accuracy, having higher latency and higher accuracy than LOHEN. Nevertheless, MS-LOHEN-L exhibits lower latency than all alternatives, including FHE-MP-CNN, and higher accuracy than LOHEN or FHE-MP-CNN. Unlike these two, four out of the six alternatives (TFHE, SHE, HEIR, and HEIR+Opt) exhibit higher inference latency ($1.09\text{--}96.1\times$) than FHE-MP-CNN or all LOHEN variants. TFHE, HEIR, and HEIR+Opt do not suffer from accuracy loss, but none of these outperform MS-LOHEN-A, which also exhibits zero accuracy loss. We also observed that our profile-guided approach led MS-LOHEN-B to find a sweet spot between the two other options.

6.3 Precision of the Latency Estimation

We evaluate the precision of LOHEN’s latency estimation by comparing the estimated cost for various workloads. We repeatedly perform our cost estimation analysis 20 times for each of the 12 NNs from Fig. 11 and Fig. 12 to compare

it with their actual latency. As Fig. 14 shows, the estimated latencies are linearly correlated with the actual latency with correlation coefficients of 0.9924 and 0.9889, suggesting that LOHEN’s latency estimation provides reliable grounds for selecting optimal operations and configuration for each layer.

7 Related Work

Reducing Single-Scheme NNs Inference Latency. FHE-MP-CNN [8], HCN [3] and multiple prior works [4, 10, 31, 32] designed techniques to reduce the inference latency of single-scheme NNs. Although our evaluation measured how LOHEN further reduces the latency when applied to FHE-MP-CNN, LOHEN can be applied to any of the aforementioned works and enhance their performance, utilizing the cryptographic repacking to adopt layer-wise CC and avoiding rotations. We also demonstrated that LOHEN successfully helps LoLat, which shares some characteristics with recent studies [10, 31] aimed at lower latency.

Computation-Adaptive Optimization. HECO [2], CHET [5], ngraph-he [7], ELASM [29], HECATE [28], COYOTE [6] introduced automated but limited mechanisms to select a global CC to run a given application. HECO, CHET, and ngraph-he select CC factors with enough levels to compute the whole application without BTS. They are shown to be effective in choosing CCs for relatively shallow models, but the lack of consideration for BTS leaves questions about how they would choose CC for deep NNs, such as ISQ or IMO. On the contrary, LOHEN does not suffer from such restrictions, takes the BTS cost into consideration, and is shown to be effective in reducing the inference latency of deep NNs. Another difference is that LOHEN selects layer-wise CC with the help of cryptographic repacking, further reducing the inference latency. Other works target certain specific parts of the CC selection to reduce the computational overhead with which LOHEN has complementary relations. COYOTE, HECO, and CHET offer automated data layout selection (*i.e.*, packing mechanisms) to enhance the overall performance by avoiding intermediate rotations during CKKS computation. On top of their work, LOHEN can improve performance by avoiding the remaining rotations by repacking. HECATE and ELASM schedule FHE operations to maximize the efficiency for a given ciphertext level limit, while DaCapo [20] schedules BTS to reduce the overhead. LOHEN can exploit their work to estimate the costs of each layer while selecting the optimal CC and operation during our cost analysis.

Multi-Scheme Computation. CHIMERA [12], PEGASUS [11], and HERMES [34] introduced and optimized the scheme conversion gadgets that enable computing on both RLWE and LWE ciphertexts (*i.e.*, CKKS, BFV, and TFHE). Glyph [39] and HEIR [13] use this conversion gadget to perform efficient yet accurate computation by using CKKS for all arithmetic and TFHE for all non-arithmetic layers with a

pre-defined CC for each scheme. LCR is built with the building blocks used for the scheme conversion gadgets, primarily to allow adaptive CC switching between RLWE ciphertexts. LOHEN also differs from the latter three in that it takes an analytic approach to adaptively use multiple CCs to enhance the overall performance and obtain a desired accuracy level. LOHEN also provides methods to exploit trade-offs between performance and accuracy, which prior works do not.

Integrating with other techniques. To manage the computational demands of FHE, hybrid approaches combining FHE with MPC have emerged for collaboratively running NNs [51]. This approach involves using FHE for arithmetic layers in the cloud and MPC for non-arithmetic layers at the client end or within a client-controlled Trusted Execution Environment (TEE) [52, 53]. This setting enables the clients to decrypt and encrypt the intermediate results from which they can perform CC switching without cryptographic algorithms [54]. This approach is more desirable if the client is willing to invest in its computing resources and participate in the computation. However, we envision other use cases where the client prefers not to be involved in the computation, where an approach relying only on FHE is the only option, and LOHEN will become a useful building block.

8 Conclusion

We have demonstrated that when FHE is applied to NNs, employing layer-wise optimization can induce NNs to perform inference over encrypted data optionally with higher performance or accuracy than previous work relying on conventional approaches. For the demonstration, we have developed a layer-wise optimization technique, LOHEN, and conducted two sets of experiments: one for performance and the other for both performance and accuracy in tandem. In the first experiment, LOHEN reduces the inference latency of CKKS-applied NNs by selectively leveraging LCR, an enhanced version of cryptographic repacking specifically for LOHEN. LCR comes with a novel gadget whose potential benefits in reducing the inference time are revealed empirically. However, this gadget has the drawback of being effective only in cases under certain conditions. Sadly, identifying such cases by hand is extremely time-consuming and error-prone since it requires extensive analysis of the structure and properties of each NN layer. LOHEN is engineered to automate this analysis, effectively deploying its gadget to enhance the performance of CKKS-applied NNs. In the second experiment, LOHEN minimizes accuracy loss to little or none, yet delivering performance comparable to previous works. This is accomplished through a layer-wise application of multi-schemes (CKKS+TFHE) coupled with a layer-wise selection of CKKS CCs. In support of LOHEN, the functionalities of LCR have been further enhanced to facilitate TFHE computations during the repacking process. Lastly, we have shown that LOHEN empowers developers to selectively apply TFHE

to NN layers in three different manners, providing them the capability to precisely adjust the accuracy and performance of NN operations at their disposal.

References

- [1] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, 2009.
- [2] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. Heco: Automatic code optimizations for efficient fully homomorphic encryption. *arXiv preprint arXiv:2202.01649*, 2022.
- [3] Ahmad Al Badawi, Chao Jin, Jie Lin, Chan Fook Mun, Sim Jun Jie, Benjamin Hong Meng Tan, Xiao Nan, Khin Mi Mi Aung, and Vijay Ramaseshan Chandrasekhar. Towards the alexnet moment for homomorphic encryption: Hcnn, the first homomorphic cnn on encrypted data with gpus. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1330–1343, 2020.
- [4] Alon Brutzkus, Ran Gilad-Bachrach, and Oren Elisha. Low latency privacy preserving inference. In *International Conference on Machine Learning*, pages 812–821. PMLR, 2019.
- [5] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. Chet: An optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 142–156, 2019.
- [6] Raghav Malik, Kabir Sheth, and Milind Kulkarni. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 118–133, 2023.
- [7] Fabian Boemer, Yixing Lao, Rosario Cammarota, and Casimir Wierzynski. ngraph-he: a graph compiler for deep learning on homomorphically encrypted data. In *Proceedings of the 16th ACM international conference on computing frontiers*, pages 3–13, 2019.
- [8] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*, pages 12403–12422. PMLR, 2022.
- [9] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*, pages 201–210. PMLR, 2016.

- [10] Jaiyoung Park, Michael Jaemin Kim, Wonkyung Jung, and Jung Ho Ahn. Aespa: Accuracy preserving low-degree polynomial activation for fast private inference. *arXiv preprint arXiv:2201.06699*, 2022.
- [11] Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073. IEEE, 2021.
- [12] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.
- [13] Song Bian, Zian Zhao, Zhou Zhang, Ran Mao, Kohei Suenaga, Yier Jin, Zhenyu Guan, and Jianwei Liu. Heir: A unified representation for cross-scheme compilation of fully homomorphic computation. *Cryptology ePrint Archive*, 2023.
- [14] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full rns variant of approximate homomorphic encryption. In *Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*, pages 347–368. Springer, 2019.
- [15] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.
- [16] Léo Ducas and Daniele Micciancio. Fhew: bootstrapping homomorphic encryption in less than a second. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 617–640. Springer, 2015.
- [17] CrypttoLab. Official HEaaN Library, 2022. <https://heaan.it/>.
- [18] Microsoft seal (release 3.7). <https://github.com/Microsoft/SEAL>, 2021.
- [19] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. Meta-bts: Bootstrapping precision beyond the limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 223–234, 2022.
- [20] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. Dacapo: Automatic bootstrapping management for efficient fully homomorphic encryption.
- [21] Zama. Concrete ML: a privacy-preserving machine learning library using fully homomorphic encryption for data scientists, 2022. <https://github.com/zama-ai/concrete-ml>.
- [22] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.
- [23] I. Chillotti et al. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning: 5th International Symposium*. Springer, 2021.
- [24] A. Sanyal et al. Tapas: Tricks to accelerate (encrypted) prediction as a service. In *International conference on machine learning*. PMLR, 2018.
- [25] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022.
- [26] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. High-throughput deep convolutional neural networks on fully homomorphic encryption using channel-by-channel packing. *Cryptology ePrint Archive*, 2023.
- [27] Miran Kim, Xiaoqian Jiang, Kristin Lauter, Elkhan Ismayilzada, and Shayan Shams. Secure human action recognition by encrypted neural network inference. volume 13, page 4799. Nature Publishing Group UK London, 2022.
- [28] Yongwoo Lee, Seonyoung Heo, Seonyoung Cheon, Shinung Jeong, Changsu Kim, Eunkyoung Kim, Dongyoon Lee, and Hanjun Kim. Hecate: Performance-aware scale optimization for homomorphic encryption compiler. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–204. IEEE, 2022.
- [29] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. {ELASM}:{Error-Latency-Aware} scale management for fully homomorphic encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4697–4714, 2023.
- [30] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. Eva: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 546–561, 2020.

- [31] Donghwan Kim, Jaiyoung Park, Jongmin Kim, Sangpyo Kim, and Jung Ho Ahn. Hyphen: A hybrid packing method and optimizations for homomorphic encryption-based neural networks. *arXiv preprint arXiv:2302.02407*, 2023.
- [32] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. High-throughput deep convolutional neural networks on fully homomorphic encryption using channel-by-channel packing. *Cryptology ePrint Archive*, 2023.
- [33] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Field switching in bgv-style homomorphic encryption. *Journal of Computer Security*, 21(5):663–684, 2013.
- [34] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. Hermes: Efficient ring packing using mlwe ciphertexts and application to transciphering. In *Annual International Cryptology Conference*, pages 37–69. Springer, 2023.
- [35] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 12403–12422. PMLR, 17–23 Jul 2022.
- [36] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. Fhe-mp-cnn.
- [37] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 360–384. Springer, 2018.
- [38] Eunsang Lee, Joon-Woo Lee, Young-Sik Kim, and Jong-Seon No. Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [39] Qian Lou, Bo Feng, Geoffrey Charles Fox, and Lei Jiang. Glyph: Fast and accurately training deep neural networks on encrypted data. *Advances in neural information processing systems*, 33:9193–9202, 2020.
- [40] Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W Keckler. Understanding error propagation in deep learning neural network (dnn) accelerators and applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2017.
- [41] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [42] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [43] Tensorsandbox, 2017. <https://github.com/dacorvo/tensorsandbox>.
- [44] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.
- [45] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- [46] DESILO. Liberate.FHE: A New FHE Library for Bridging the Gap between Theory and Practice with a Focus on Performance and Accuracy, 2023. <https://github.com/Desilo/liberate-fhe>.
- [47] SNU CNL. Fhe-mp-cnn, 2021. <https://github.com/snu-ccl/FHE-MP-CNN>.
- [48] A gpu implementation of fully homomorphic encryption on torus. <https://github.com/nucypher/nufhe>.
- [49] François Chollet. Keras. <https://keras.io>, 2015.
- [50] Qian Lou and Lei Jiang. She: A fast and accurate deep neural network for encrypted data. *Advances in Neural Information Processing Systems*, 32, 2019.
- [51] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. {GAZELLE}: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1651–1669, 2018.
- [52] Ankur Dave, Chester Leung, Raluca Ada Popa, Joseph E Gonzalez, and Ion Stoica. Oblivious cooperative analytics using hardware enclaves. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–17, 2020.

- [53] Olga Ohrimenko, Felix Schuster, Cédric Fournet, Aastha Mehta, Sebastian Nowozin, Kapil Vaswani, and Manuel Costa. Oblivious multi-party machine learning on trusted processors. In *USENIX Security Symposium*, volume 16, pages 10–12, 2016.
- [54] Qian Lou, Song Bian, and Lei Jiang. Autoprivacy: Automated layer-wise parameter selection for secure neural network inference. *Advances in Neural Information Processing Systems*, 33:8638–8647, 2020.

Algorithm 2: Ringswitch (to a lower degree) [34]

Input: $ct_{in} \in RLWE_{q,N}$, $swk_{s \rightarrow s'}$ **Output:** $(ct_{out}^i)_{0 \leq i < \frac{N}{n}} \in RLWE_{q,n}$

- 1 $ct' \leftarrow KS(ct_{in}, swk_{s \rightarrow s'})$
 - 2 $(ct_{out}^i)_{0 \leq i < \frac{N}{n}} \leftarrow Split(ct')$
 - 3 **return** $(ct_{out}^i)_{0 \leq i < \frac{N}{n}}$
-

Algorithm 3: LWE-Extract [11]

Input: $ct_{in} \in RLWE_{q,n}$ **Output:** $(ct_i)_{0 \leq i < n} \in LWE_q^n$

- 1 **for** $i = 0$ **to** $n - 1$ **do**
 - 2 $ct_i \leftarrow Extract^i(ct_{in})$
 - 3 **return** $(ct_i)_{0 \leq i < n}$
-

Algorithm 4: RLWE-Pack [34]

Input: $(ct_{in}^i)_{0 \leq i < n} \in LWE_q^n$, $(swk_{s' \rightarrow s}^i)_{0 \leq i < n}$ **Output:** $ct_{out} \in RLWE_{q,n}$

- 1 Let $ct_{in}^i = (b_i, a_i^0, a_i^1, \dots, a_i^{n-1}) \in \mathbb{Z}_q \times \mathbb{Z}_q^n$
 - 2 $\alpha_j(X) \leftarrow \sum_{i=0}^{n-1} a_i^j X^i$ for all $j < n$
 - 3 $ct_{out} \leftarrow \text{modDown}(\sum_{j=0}^{n-1} \text{modUp}(\alpha_j(X)) \cdot swk_{s' \rightarrow s}^j) + (0, \sum_{i=0}^{n-1} b_i X^{n-1})$
 - 4 **return** ct_{out}
-

A CC Switching Routines & LCR

LCR algorithm enables LOHEN to switch between CKKS CCs by combining three key CC switching routines as described in §4.2. We present a more detailed exhibition of the routines, including their references, as well as how they are used to build LCR.

Algo. 2 shows an algorithm of Ringswitch RLWE ciphertext in degree N to smaller n . Ringswitch from a large ring to smaller rings consists of key-switching to a small-degree key s' and Split. In the opposite direction, from a smaller rings to a larger ring, the process includes Combine, followed by key-switching to a large-degree key s . Both the functions Split and Combine rearrange the coefficients to produce ciphertexts with small and large degrees, respectively.

LWE-Extract brings out i -th coefficient of RLWE ciphertext, which contains the i -th message as shown in Algo. 3. The function $Extract^i$ takes RLWE ciphertext (b, a) as an input and extracts i -th LWE ciphertext $(b_i, a^{(i)})$ where $a^{(i)} = (a_i, a_{i-1}, \dots, a_0, -a_{i-1}, \dots, -a_{i+1})$. Note that this process simply rearranged the values, incurring minimal additional cost.

Algo. 4 depicts RLWE-Pack that comprises of three sub-operations: modDown, modUp and hadamard multiplication, which are used in the key switching operation. modUp turns a polynomial in $R_{q,N}$ to R_N and applies modulo pq , modDown is an approximate division by p which maps polynomial a

into $(a - [a]_p)/p$. RLWE-Pack first regards given n LWE ciphertexts as a matrix $M \in \mathbb{Z}_q^{n \times n}$ where each rows are the LWE ciphertexts. It defines j -th column of M as $\alpha_j(X)$ and then sums the results of modUp of $\alpha_j(X)$ and applies hadamard multiplication with switching key and modDown, ends up with adding b . Consequently, we obtain the RLWE ciphertext that holds the message of LWE ciphertexts in the given order.

The full algorithm of LCR is outlined in Algo. 5. It begins with StC to convert slot-encoded ciphertext into a coefficient-encoded format (line 1), placing each slot element at the position of coefficients. Then, we decrease the ring degree to lower the computational cost of LWE operations and generate N/n RLWE ciphertexts in the lower ring degree by switching key and splitting coefficients (line 2). Following the ring switch, we extract n LWE ciphertexts from each N/n RLWE ciphertexts (line 3-line 5). During the Permute (line 6), LOHEN performs not only permutation given LWE ciphertexts but also evaluates TFHE-based non-linear operations. After Applying RLWE-Pack on the re-ordered N/n groups of n LWEs, which generates n RLWEs in ring degree n , we combine these n RLWEs and switch the ring back to N (line 7-line 9). Finally, We employ three sequential routines to recover the modulus, which follows the same process as in BTS (line 10).

B Benchmark of FHE operations

As discussed in §6, we do not use the same library as FHE-MP-CNN or Hyphen for several major reasons. Here, our evaluations are fair since we run all baselines with the same implementation as LOHEN, and we list several empirical reasons why we made such decisions in experimental settings. Table 16 shows the difference of the SEAL variant used in FHE-MP-CNN and HEaAN on CPU. Note that while other operations show similar performance, the latency of BTS differs on a great scale. FHE-MP-CNN reports 80% of their latency to be BTS, which seems to be related to this profiling. If they were run with HEaAN, their bottleneck would not have been the GPU, which we can deduce with Table 16 and Table 7. Note that LOHEN's effectiveness grows when BTS time is bigger, thus this application on GPU is in fact in the favor of the compared baselines, not LOHEN.

It is also noteworthy that HEaAN GPU takes 354,762 us for BTS when $n = 2^{17}$ [20] while Liberate-FHE takes 371,541. The numbers are similar (Liberate-FHE being a bit slower), suggesting that the works that use HEaAN GPU can be compared to be experimented under similar conditions with our work.

StC is always performed at a level 3 and consumes three moduli, resulting in a ciphertext at a level 0; thus, it shows constant latency dependent on the N (i.e., 641 ms for $N = 2^{16}$). To recover the modulus following RLWE-Pack, the operations ModRaise, CtS, and EvalMod must be executed in sequence. The cost of CtS and EvalMod is $8.86 \times$, $5.22 \times$ higher than that of StC respectively since Those are performed at a higher

Algorithm 5: LCR

Input: $ct_{in} \in RLWE_{q,N}$
Output: $ct_{out} \in RLWE_{Q,N}$

```

1  $ct' \in RLWE_{q',N} \leftarrow \text{StC}(RLWE_{q,N})$ ; // convert to coeff-encoded
2  $\{ct''_0, ct''_1, \dots, ct''_{\frac{N}{n}-1}\} \in RLWE_{q',n} \leftarrow \text{Split}(\text{KS}(ct', \text{swk}_{s \rightarrow s'}))$ ; // ring switch to lower degree of ciphertexts
3 for  $i = 0$  to  $\frac{N}{n} - 1$  do
4   for  $j = 0$  to  $n - 1$  do
5      $ct'''_{i \cdot n + j} \in LWE_{q'}^n \leftarrow \text{Extract}^j(ct''_i)$ ; // extract LWE ciphertexts
6 Permute( $ct'''$ ); // Re-ordering of LWE ciphertexts
7 for  $i = 0$  to  $\frac{N}{n} - 1$  do
8    $ct''''_i \in RLWE_{q',n} \leftarrow \text{RLWE-Pack}((ct'''_j)_{i \cdot n \leq j < (i+1) \cdot n})$ ; // RLWE-pack
9  $\tilde{ct} \in RLWE_{q',N} \leftarrow \text{KS}(\text{Combine}((ct''''_i)_{0 \leq i < \frac{N}{n}}, \text{swk}_{s' \rightarrow s}))$ ; // ring switch to higher degree of ciphertexts
10  $ct_{out} \in RLWE_{Q,N} \leftarrow \text{EvalMod}(\text{CtS}(\text{ModRaise}(\tilde{ct})))$ ; // Mod recover
11 return  $ct_{out}$ 

```

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

(a) Convolution in plaintext

$$\begin{bmatrix} \# & \# & \# \\ \# & a_{11} & a_{12} \\ a_{13} & a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 0 \\ 0 & f_{11} & f_{12} \\ 0 & f_{11} & f_{12} \end{bmatrix} + \begin{bmatrix} \# & \# & \# \\ a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 0 \\ f_{12} & f_{12} & f_{12} \\ f_{12} & f_{12} & f_{12} \end{bmatrix} + \dots = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

(b) Convolution in ciphertext

Fig. 15: Example of SISO convolution $(f, st, p) = (3, 1, 1)$

Table 16: SEAL [18] and HEaaN [17] on CPU (s) when $N = 2^{16}$, $lvl = 7$, $Q = 768$

seconds	BTS	Cmult	Pmult	Rotate<<1
SEAL	120	0.17	0.002	0.17
HEaaN	9.91	0.15	0.002	0.16

level following ModRaise (*i.e.*, 5,682, 3,346 ms for identical N). The cost of EvalMod is relatively negligible, as it does not involve homomorphic evaluation.

C Routines In Bootstrapping procedure

In the bootstrapping procedure, there are four steps, each of which is vital for the repacking procedure. We provide a detailed algorithmic explanation for the necessity of each step in our repacking algorithm. There are routines facilitating the transition of encoding states and operations such as ModRaise, which increases the ciphertext modulus, and EvalMod, which approximately performs modulo operations. **StC.** In preprocessing for CC Switching routines, we need a coefficient-encoded ciphertext to correctly switch the CC, keeping the value of the ciphertext. To convert a slot-encoded

ciphertext into a coefficient ciphertext, we need to apply decoding functions homomorphically to the slots of the ciphertext. StC routine is the function which is homomorphic evaluation of the decoding function of CKKS (*i.e.*, evaluation of discrete Fourier transform(DFT) matrix U homomorphically), converting a slot-encoded $RLWE((m(X)))$ into a coefficient-encoded $RLWE(\Delta \cdot \mathbf{m})$.

The DFT matrix $\mathbf{U}$ is expressed as follows such that $m(\zeta_i) = \mathbf{m}_i, \forall i \in [0, \frac{N}{2} - 1]$

$$\mathbf{U} = \begin{pmatrix} 1 & \zeta_0 & \dots & \zeta_0^{N-1} \\ 1 & \zeta_1 & \dots & \zeta_1^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{\frac{N}{2}-1} & \dots & \zeta_{\frac{N}{2}-1}^{N-1} \end{pmatrix}$$

CtS. CtS is the inverse routine of StC which convert the coefficient-encoded state to the slot-encoded. After executing a repacking algorithm on coefficient-encoded ciphertext, it is necessary to revert to slot-encoded ciphertext in order to continue the HE evaluation. CtS routine is a evaluation of inverse DFT(iDFT) matrix homomorphically (*i.e.* homomorphically evaluating encoding function). We can execute a CtS routine immediately after the repacking algorithm, but we can also apply ModRaise earlier and EvalMod later to extend the ciphertext modulus, enabling the execution of additional homomorphic evaluations.

ModRaise & EvalMod. The routines ModRaise and EvalMod are exclusively employed for recovering the ciphertext modulus. Following the RLWE-pack process, the CtS routine can be immediately executed to transform the coefficient-encoded ciphertext into a slot-encoded ciphertext. However, in scenarios where a low level is encountered after the RLWE-pack, we decide to execute an additional two routines with CtS to recover the modulus.

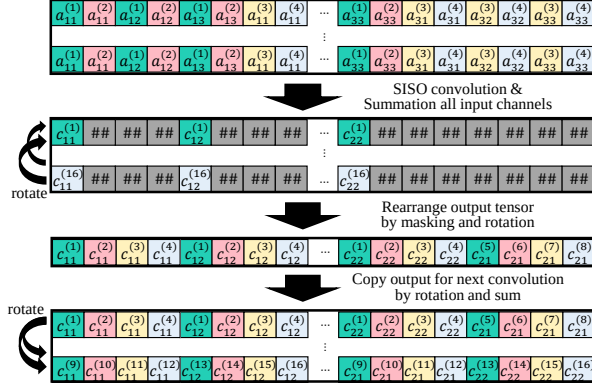


Fig. 16: Example of multiplexed convolution

D Dynamic Rotation Costs

The number of rotations in NN layers is determined by the used CC as well as the computing mechanism. In the case of convolutions, several works introduce their own methods with specific goals, such as minimizing the number of multiplications or the number of ciphertexts. According to the specific method, the number of rotations can also be determined. As our work focuses on evaluating LOHEN over FHE-MP-CNN, we give an example of how we can determine the number of rotations.

Multiplexed parallel convolution [8] utilizes Single-Input-Single-Output (SISO) convolution proposed in Gazelle [51]. Before describing details, SISO convolution begins with an encrypted input tensor consisting of elements in a raster-scan fashion as depicted in Fig. 15. By rotating the ciphertext to

match the corresponding position of output and multiplying filter values, this process yields the result of convolution. Consequently, SISO convolution requires $f_h \times f_w - 1$ operations for both multiplication and rotation.

Fig. 16 illustrates an example of multiplexed parallel convolution, which aims to maximize the utilization of ciphertext slots by packing multiple channels compactly. Each ciphertext between layers holds multiple copies of the intermediate tensor, enabling SISO convolution for multiple output channels in parallel. For example, a tensor $h_i \times w_i \times c_i = 8 \times 8 \times 16$ holds 1,024 elements which can be placed in a ciphertext with $N = 2^{14}$. In multiplexed parallel convolution, the tensor is copied up to 8 times so that the ciphertext is fully packed, which enables running 8 SISO convolution in parallel. For a tensor with multiple output channels, the result elements of SISO convolution and summation input channels are sparsely placed in a ciphertext. To improve slot utilization, scattered elements are densely packed in a single ciphertext again by masking the valid values and rotating. Throughout multiplexed parallel convolution for multiple channels, the number of rotation operation is $f_h \times f_w - 1 + \lceil \frac{c_o}{p_i} \rceil (2 \lceil \log_2(k_i) \rceil + \lceil \log_2(\frac{c_i}{k_i^2}) \rceil) + c_o + \log_2 p_o$, where $p_o = 2^{\lceil \log_2(\frac{N}{2 \times k_o^2 h_o w_o t_o}) \rceil}$ means the number of input tensor copies considering a gap of input ciphertext. Note that the number is relevant to the total slots of ciphertext, $N/2$. Since this convolution methodology assumes all elements of input tensors can be packed into a single ciphertext, we extend the same approach even if the ciphertext is not able to hold all elements. We used this information for our cost estimation analysis proposed in §4.5.